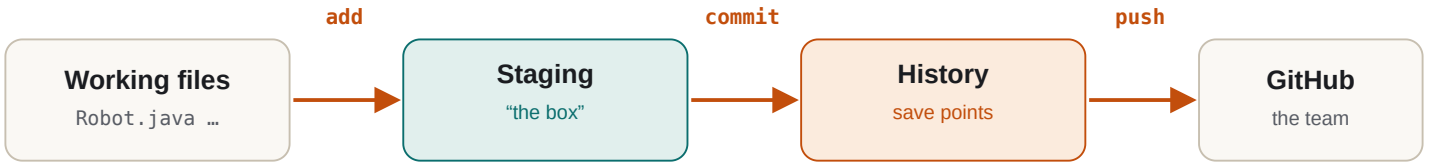
Git in Four Commands

Git takes **labeled snapshots** of your project — save points you can always return to. Saving a file (Ctrl+S) writes it to disk now; a **commit** is a separate, labeled snapshot.

FRC 1721
WEEK 1 HANDOUT



`init = turn on tracking (once)`

THE FOUR COMMANDS

`git init` Start tracking — `once per project`

Turns on version history for this folder (creates the hidden `.git`). Nothing saved yet — like switching on a camera that hasn't recorded.

`git add .` Pack the box — `before every commit`

Stage the changes you want in the next snapshot. The `.` means “everything that changed.” You're choosing what goes in the box **before** taping it shut.

`git commit -m "…"` Seal & label — `your save point`

Permanently records what you staged, with a message describing it — e.g. `-m "Week 1: Hello, Robot"`. That label is how you find it later.

`git push` Send to the team — `shares your work`

Uploads your commits to GitHub so teammates see them. `init` / `add` / `commit` all happen on your laptop first.

ONE-TIME SETUP

Tell Git who you are

```
git config --global user.name "Your Name"
git config --global user.email "you@email.com"
```

EVERYDAY FLOW

`add .` → `commit -m` → `push`

Repeat this loop each time you finish something worth saving.

WATCH OUT

- **“nothing to commit”** → you forgot `git add .` (empty box).
- A commit **needs a message** — always use `-m "…"`.
- `git init` is **one time only** per project.
- Auth error on `push`? Get your **commit** done first; finish push with a mentor.

`git status`
what's changed right now

`git log --oneline`
your save points, listed